



12

Epilogue: Principles of Algorithms

While we have reached the end of this book, there is no limit to the amount of information you can find on algorithms about which you're interested. Indeed, there is no end to the kind of problems to which you can apply the techniques presented in this book.

We finally have the opportunity to step back and review the nearly three dozen algorithms we described in detail and by example. We hope you agree that we have accomplished what we set out to do. To show the breadth of material that we've covered, we now summarize the principles behind the algorithms presented in this book. In doing so, we can demonstrate the similarities of different algorithms that were designed to solve different problems. Instead of simply summarizing each of the previous chapters, we'll end this book by focusing on key principles that were instrumental in designing these algorithms in the first place. We also take this opportunity to summarize the concepts used by each algorithm. Thus, we provide a quick summary and make it possible to cross-index this book in terms of shared concepts across different algorithms.

Know Your Data

This book discussed a variety of common actions you might need to perform on some data. You might need to sort data to produce a specific ordering. You might need to search through data to locate a specific piece of information. Your data may be accessible in random access (where you can fetch any piece of information at any time) or sequentially using an Iterator (where each element is generated one at a time). Without specific knowledge about your data, it is only possible to recommend algorithms in the most general way.

Often properties about the input data have a significant impact. In [Chapter 9](#), many special cases can simply be eliminated if you know you are computing the intersec-

tions among line segments containing no vertical lines; similarly, computing the Voronoi diagram is simplified if no two points share the same x or y coordinate. **Dijkstra's Algorithm**, found in [Chapter 6](#), will run forever if a cycle exists whose sum of all edge weights is negative. Make sure you understand the special cases and assumptions of the algorithms you choose.

As we have argued, there is no single algorithm that consistently delivers the best performance for all circumstances. Choose the most appropriate algorithm based on your data as you become more familiar with the available options. [Table 12-1](#) summarizes the results of the sorting algorithms presented in [Chapter 4](#). Naturally you will focus on the worst-case performance of each algorithm, but also pay attention to the *concepts* that arise when implementing or using these algorithms.

Table 12-1. Sorting algorithms

Algorithm	Best	Average	Worst	Concepts	Page
Bucket Sort	n	n	n	Hash	"Bucket Sort" on page 74
Heap Sort	$n \log n$	$n \log n$	$n \log n$	Recursion, Binary Heap	"Heap Sort" on page 62
Insertion Sort	n	n^2	n^2	Greedy	"Transposition Sorting" on page 57
Merge Sort	$n \log n$	$n \log n$	$n \log n$	Recursion, Stable, Divide and Conquer	"Merge Sort" on page 81
Quicksort	$n \log n$	$n \log n$	n^2	Recursion, Divide and Conquer	"Partition-Based Sorting" on page 67
Selection Sort	n^2	n^2	n^2	Greedy	"Selection Sort" on page 61

Decompose a Problem into Smaller Problems

When designing an efficient algorithm to solve a problem, it is helpful if the problem can be decomposed into two (or more) smaller subproblems. It is no mistake that **Quicksort** remains one of the most popular sorting algorithms. Even with the well-documented special cases that cause problems, **Quicksort** offers the best average-case for sorting large collections of information. Indeed, the very concept of an $O(n \log n)$ algorithm is based on the ability to (a) decompose a problem of size n into two subproblems of about $n/2$ in size, and (b) recombine the solution of the two subproblems into a solution for the original problem. To design an $O(n \log n)$ algorithm, it must be possible for both of these steps to execute in $O(n)$ time.

Quicksort was the first in-place sorting algorithm to demonstrate $O(n \log n)$ performance. It succeeds by the novel (almost counterintuitive) approach for dividing the problem into two halves, each of which can be solved recursively by applying **Quicksort** to the smaller subproblems.

Problems often can be simply cut in half, leading to impressive performance savings. Consider how **Binary Search** converts a problem of size n into a problem of

size $n/2$. **Binary Search** takes advantage of the repetitive nature of the search task to develop a recursive solution to the problem.

Sometimes a problem can be solved by dividing it into two subproblems without resorting to recursion. **Convex Hull Scan** produces the final convex hull by constructing and merging together two partial hulls (the upper and lower).

Sometimes a problem can be decomposed into the repeated iteration of a different (seemingly unconnected) smaller problem over the same input data. **Ford–Fulkerson** computes the maximum flow in a flow network by repeatedly locating an augmenting path to which flow can be added. Eventually, no augmenting paths are possible and the original solution is solved. **Selection Sort** repeatedly locates the maximum value in an array and swaps it with the rightmost element in the array; upon completing n iterations, the array is sorted. Similarly, **Heap Sort** repeatedly swaps the largest element in the heap with its proper location in the array.

Observe that Dynamic Programming decomposes problems into smaller problems, but its overall behavior is typically $O(n^2)$ or $O(n^3)$ because the smaller problems are typically just one size smaller than the original problem, rather than half its size.

Table 12-2 contains a comparison of the searching algorithms discussed in **Chapter 5**. These algorithms offer different approaches to answer the fundamental question of membership in a collection. In analyzing their performance, we used a technique to *amortize* the costs of a series of operations, which allows us to accurately characterize the *average* performance given a random search query.

Table 12-2. Searching algorithms

Algorithm	Best	Average	Worst	Concepts	Page
AVL Binary Search Tree	1	$\log n$	$\log n$	Binary Tree, Balanced	“Binary Search Tree” on page 119
Sequential Search	1	n	n	Brute Force	“Sequential Search” on page 92
Binary Search	1	$\log n$	$\log n$	Divide and Conquer	“Binary Search” on page 95
Bloom Filter	k	k	k	False Positive	“Bloom Filter” on page 114
Hash-Based Search	1	1	n	Hash	“Hash-Based Search” on page 99
Binary Search Tree	1	$\log n$	n	Binary Tree	“Binary Search Tree” on page 119

Choose the Right Data Structure

The famed algorithm designer Robert Tarjan was once quoted as saying that any problem can be solved in $O(n \log n)$ time with the right data structure. Many algorithms need to use a priority queue to store partial progress and direct future computations. One of the most common means of implementing a priority queue is through a binary heap, which allows for $O(\log n)$ behavior for removing the element with lowest priority from the priority queue. However, a binary heap offers no ability to determine whether it contains a specific element. We expanded on this very point in the discussion of **LineSweep** (**Chapter 9**). This algorithm can provide

$O(n \log n)$ performance because it uses an augmented binary tree to implement the priority queue and still provides $O(\log n)$ performance for removing the minimum element. Another way to state this principle is to avoid selecting an inappropriate data structure that will prevent an algorithm from achieving its best performance.

In [Chapter 6](#), we showed when to use an adjacency list or an adjacency matrix to represent a graph, based on whether the graph was *sparse* or *dense*. This single decision has the greatest impact on the performance of these algorithms. [Table 12-3](#) shows the graph algorithms discussed in [Chapter 6](#).

Table 12-3. Graph algorithms

Algorithm	Best	Average	Worst	Concepts	Page
Bellman-Ford Algorithm	V^*E	V^*E	V^*E	Weighted Directed Graph, Overflow	“All-Pairs Shortest Path” on page 159
Breadth-First Search	$V + E$	$V + E$	$V + E$	Graph, Queue	“Breadth-First Search” on page 143
Depth-First Search	$V + E$	$V + E$	$V + E$	Graph, Recursion, Backtracking	“Depth-First Search” on page 137
Dijkstra’s Algorithm PQ	$(V + E) \log V$	$(V + E) \log V$	$(V + E) \log V$	Weighted Directed Graph, Priority Queue, Overflow	“Single-Source Shortest Path” on page 147
Dijkstra’s Algorithm DG	$V^2 + E$	$V^2 + E$	$V^2 + E$	Weighted Directed Graph, Overflow	“Single-Source Shortest Path” on page 147
Floyd–Warshall Algorithm	V^3	V^3	V^3	Dynamic Programming, Weighted Directed Graph, Overflow	“All-Pairs Shortest Path” on page 159
Prim’s Algorithm	$(V + E) \log V$	$(V + E) \log V$	$(V + E) \log V$	Weighted Graph, Binary Heap, Priority Queue, Greedy	“Minimum Spanning Tree Algorithms” on page 163

When working with complex n -dimensional data, you need more complicated recursive structures to store the data. [Chapter 10](#) describes sophisticated *spatial tree* structures to efficiently support standard search queries as well as more complicated range queries. These structures were carefully designed by extending *binary trees*, the fundamental recursive data structure in computer science.

Make the Space versus Time Trade-Off

Many of the computations carried out by the algorithms are optimized by storing information that reflects the results of past computations. **Prim's Algorithm** for computing the minimum spanning tree for a graph uses a priority queue to store the unvisited vertices in order of their shortest distance to an initial vertex s . During a key step in the algorithm, we must determine whether a given vertex has already been visited. Because the binary heap implementation of the priority queue fails to provide this operation, a separate Boolean array records the status of each vertex. In the same algorithm, another array stores the computed distances to avoid having to search again through the priority queue. This extra storage on the order of $O(n)$ is required to ensure the efficient implementation of the algorithm. In most situations, as long as the overhead is $O(n)$, you are going to be safe.

Sometimes an entire computation can be cached so it never needs to be recomputed. In [Chapter 6](#), we discussed how the hash function for the `java.lang.String` class stores the computed hash value to speed up its performance.

Sometimes the nature of the input set demands a large amount of storage, such as the dense graphs described in [Chapter 6](#). By using a two-dimensional matrix to store the edge information—rather than using simple adjacency lists—certain algorithms exhibit reasonable performance. You may also note that for undirected graphs, the algorithms can be simplified by having twice as much storage and use a two-dimensional matrix to store information for `edgeInfo[i][j]` as well as `edgeInfo[j][i]`. It would be possible to eliminate this extra information if we always queried for `edgeInfo[i][j]` using $i \leq j$, but this would further complicate every algorithm that simply desired to know whether edge (i, j) exists.

Sometimes an algorithm is unable to operate without some higher-than-expected storage. **Bucket Sort** can sort in linear time simply by storing up to $O(n)$ extra storage, if the input set is uniformly distributed. Given that today's modern computers often have very large random access memory present, you should consider **Bucket Sort** for uniform data even though its memory requirements are so high.

[Table 12-4](#) shows the spatial tree algorithms discussed in [Chapter 10](#).

Table 12-4. Spatial Tree algorithms

Algorithm	Best	Average	Worst	Concepts	Page
Nearest Neighbor Query	$\log n$	$\log n$	n	k -d tree, Recursion	"Nearest Neighbor Summary" on page 290
Quadtree	$\log n$	$\log n$	$\log n$	Quadtree	"Quadtree" on page 286
Range Queries	$n^{1-1/d} + r$	$n^{1-1/d} + r$	n	k -d tree, Recursion	"Range Query" on page 298
R-Tree	$\log n$	$\log n$	$\log n$	R-Tree	"R-Tree" on page 287

Construct a Search

Early pioneers in the field of artificial intelligence (AI) were often characterized as trying to solve problems for which no known solution existed. One of the most common approaches to solving problems was to convert the problem into a search over a very large graph. We dedicated an entire chapter to this approach because it is an important and general technique for solving numerous problems. But be careful to apply it only when no other computational alternative is available! You could use the path-finding approach to discover a sequence of element transpositions that starts from an unsorted array (the initial node) and produces a sorted array (the goal node), but you shouldn't use an algorithm with exponential behavior because numerous $O(n \log n)$ algorithms exist to sort data.

Table 12-5 shows the path-finding algorithms discussed in Chapter 7. These all exhibit exponential performance, but these are still the preferred approach for implementing intelligent game-playing programs. While these algorithms identify the structure for finding a solution, they succeed because of sophisticated heuristics that truly make the search process intelligent.

Table 12-5. Path finding in AI

Algorithm	Best	Average	Worst	Concepts	Page
Depth-First Search	b^*d	b^d	b^d	Stack, Set, Backtracking	"Depth-First Search" on page 192
Breadth-First Search	b^d	b^d	b^d	Queue, Set	"Breadth-First Search" on page 198
A*Search	b^*d	b^d	b^d	Priority Queue, Set, Heuristics	"A*Search" on page 201
Minimax	b^{ply}	b^{ply}	b^{ply}	Recursion, Backtracking, Brute Force	"Minimax" on page 174
NegMax	b^{ply}	b^{ply}	b^{ply}	Recursion, Backtracking, Brute Force	"NegMax" on page 180
AlphaBeta	$b^{ply/2}$	$b^{ply/2}$	b^{ply}	Recursion, Backtracking, Heuristics	"AlphaBeta" on page 183

Reduce Your Problem to Another Problem

Problem reduction is a fundamental approach used by computer scientists and mathematicians to solve problems. As a simple example, suppose you wanted to locate the fourth largest element in a list. Instead of writing this special-purpose code, you could use any sorting algorithm to sort the list and then return the fourth element in the sorted list. Using this approach, you have defined an algorithm whose performance time is $O(n \log n)$, although this is not the most efficient way to solve the problem.

When using **Fortune Sweep** to compute the Voronoi diagram, the convex hull can be readily computed by finding those points that share an infinite Voronoi edge within their polygons. In this regard, the algorithm computes more information than necessary, but the output can be used for a number of interesting problems, such as computing a planar triangulation of the points in the collection.

Chapter 8 presented a set of problems that all seemed related, but there didn't seem to be any easy way to tie them all together. It is possible to reduce all of these problems to linear programming (LP) and use commercially available software packages, such as Maple, to compute solutions. However, the reductions are complicated, and the general-purpose algorithms used to solve LP problems can be outperformed, often significantly, by the **Ford–Fulkerson** family of algorithms. We show in **Chapter 8** how to solve a single problem type, namely computing the minimum-cost maximum flow in a flow network. With this algorithm in hand, the five other problems are immediately solved. **Table 12-6** shows the network flow algorithms described in **Chapter 8**.

Table 12-6. Network flow algorithms

Algorithm	Best	Average	Worst	Concepts	Page
Ford–Fulkerson	E^*mf	E^*mf	E^*mf	Weighted Directed Graph, Greedy	“Maximum Flow” on page 220
Edmonds–Karp	V^*E^2	V^*E^2	V^*E^2	Weighted Directed Graph, Greedy	“Maximum Flow” on page 220

Writing Algorithms Is Hard—Testing Algorithms Is Harder

Because the algorithms we describe are predominantly deterministic (except for those from **Chapter 11**), it was rather straightforward to develop test cases to ensure they behaved properly. In **Chapter 7**, we began to encounter difficulties because we were using path-finding algorithms to locate potential solutions that we did not know in advance. For example, although it was straightforward to write test cases to determine whether the `GoodEvaluator` heuristic was working properly for the 8-puzzle, the only way to test an **A*Search** using that heuristic is to invoke the search and manually inspect the explored tree to validate that the proper move was selected. Thus, testing **A*Search** is complicated by having to test the algorithm in the context of a specific problem and heuristic. We have extensive test cases for the path-finding algorithms, but in many cases they exist only to ensure a reasonable move was selected (for either game or search trees), rather than to ensure a specific move was selected.

Testing the algorithms in **Chapter 9** was further complicated because of floating-point computations. Consider our approach to test **Convex Hull Scan**. The original idea was to execute the brute-force **Slow Hull** algorithm, whose performance was $O(n^4)$, and compare its output with the output from Andrew's **Convex Hull Scan**.

During our extensive testing, we randomly generated two-dimensional data sets uniformly drawn from the $[0, 1]$ unit square. However, when the data sets grew sufficiently large, we invariably encountered situations where the results of the two algorithms were different. Was there a subtle defect exposed by the data, or was something else at work? We eventually discovered that the floating-point arithmetic used by **Slow Hull** produced slightly (ever so slightly) different results when compared with **Convex Hull Scan**. Was this just a fluke? Unfortunately, no. We also noticed that the **LineSweep** algorithm produced slightly different results when compared with the **Brute-Force Intersection** algorithm.

Which algorithm produced the “right” result? It’s not that simple, as using floating-point values led us to develop a consistent notion of comparing floating-point values. Specifically, we (somewhat) arbitrarily defined `FloatingPoint.epsilon` to be the threshold value below which it becomes impossible to discern differences between two numbers. When the resulting computations lead to values near this threshold (which we set to 10^{-9}), unexpected behavior would often occur. Eliminating the threshold entirely won’t solve the problem, either. We ultimately resorted to statistically checking the results of these algorithms, rather than seeking absolute and definitive answers for all cases.

Table 12-7 summarizes the algorithms presented in **Chapter 9**. Each algorithm shares the challenges in working with two-dimensional geometric structures and accurately performing geometric computations.

Table 12-7. Computational geometry

Algorithm	Best	Average	Worst	Concepts	Page
Convex Hull Scan	n	$n \log n$	$n \log n$	Greedy	“Convex Hull Scan” on page 250
LineSweep	$(n + k) \log n$	$(n + k) \log n$	n^2	Priority Queue, Binary Tree	“LineSweep” on page 259
Voronoi Diagram	$n \log n$	$n \log n$	$n \log n$	LineSweep, Priority Queue, Binary Tree	“Voronoi Diagram” on page 268

Accept Approximate Solutions When Possible

In many circumstances, an approximate result is acceptable if it can be computed much faster than an accurate result and it has a known error from the correct result. The **Knapsack unbounded** problem provides such a scenario, since the approximation is no worse than 50% of the actual result. These approximations can use randomness to compute an estimate of an actual answer, as we saw with the example for counting the number of solutions to the N-Queens Problem. Use this approach when you know that repeated trials increase the precision of the estimate.

A **Bloom Filter** is carefully designed so it can return false positives, but never false negatives, when searching for an element in a collection. At first glance, it may seem

useless to have an algorithm that returns an incorrect answer. But a **Bloom Filter** can dramatically reduce the execution time of searching algorithms involving secondary storage or database systems. When it returns negative, it truly means the element does not exist in the collection, so there is no need to pursue a more costly search. Of course, it might mean that sometimes the **Bloom Filter** allows a search to continue that will fail, but this won't affect the correctness of the overall application.

Add Parallelism to Increase Performance

The algorithms presented in this book compute their results assuming a single, sequential computer. If you can identify subproblems that can be independently computed, you might be able to design a multithreaded solution using the available resources provided by modern computers. For instance, [Chapter 11](#) showed how to parallelize **Quicksort** to achieve a nice speedup. What other algorithms in this book can benefit from parallelism? Recall that **Convex Hull Scan** has a sorting substep followed by two independent problems: constructing the lower partial hull and the upper partial hull. Each of these tasks can be parallelized to achieve improved performance. [Table 12-8](#) shows the impressive speedup (review the `algs.model.problems.convexhull.parallel` code in the repository). Despite the impressive performance, the algorithm still performs in $O(n \log n)$ time, although with better constants.

Table 12-8. Performance improvements of multithreaded Convex Hull Scan

n	single-threaded	1 helper thread	2 helpers	3 helpers	4 helpers
2,048	0.8571	0.5000	0.6633	0.5204	0.6020
4,096	1.5204	0.7041	0.7041	0.7755	0.7857
8,192	3.3163	0.9592	1.0306	1.0306	1.0816
16,384	7.3776	1.6327	1.6327	1.5612	1.6939
32,768	16.3673	3.0612	2.8980	2.9694	3.1122
65,536	37.1633	5.8980	6.0102	6.0306	6.0408
131,072	94.2653	13.8061	14.3776	14.1020	14.5612
262,144	293.2245	37.0102	37.5204	37.5408	38.2143
524,288	801.7347	90.7449	92.1939	91.1633	91.9592
1,048,576	1890.5612	197.4592	198.6939	198.0306	200.5612

Most serial algorithms cannot achieve theoretic maximal speedup because only part of the algorithm can be parallelized among multiple threads; this is known as *Amdahl's law*. Don't try to use as many threads as possible in a solution. Adding multiple helper threads requires more complicated programs than adding a single helper thread. So with only a small increase in complexity, using a single helper thread can provide noticeable improvement.

However, not every algorithm can be improved with parallelism. In the k -d tree **Nearest Neighbor**, for example, there may be double recursions as the algorithm seeks to find the closest point in the collection to a target point. Parallelizing these separate method invocations will slow down the overall performance because of the need to synchronize these helper threads so they both complete together.